USER INTERFACE DESIGN

EXPERIMENT – 3

Name: ROHITH KUMAR S

ROLL NO:240701436

Develop and compare CLI, GUI, and Voice User Interfaces (VUI) for the same task and assess user satisfaction using Python (Tkinter for GUI, Speech Recognition for VUI), Terminal

Title : Multi-Interface Duplicate File Detection System Using CLI, GUI, and Voice User Interface

1.COMMAND-LINE INTERFACE:

Description:

The CLI version allows users to interact with the duplicate file detection system through text-based commands in the terminal. Users provide the folder path manually or as a command-line argument, and the system scans the directory recursively to detect duplicate files.

Functionality:

- Accepts folder path through terminal input.
- Recursively scans directories using os.walk.
- Generates file hashes using the MD5 algorithm.
- Displays original and duplicate file paths clearly.
- Prompts the user for confirmation before deletion.
- Deletes duplicate files while keeping the original file intact.
- Provides a fast and lightweight execution environment.

Advantage:

The CLI is efficient for technical users and consumes minimal system resources.

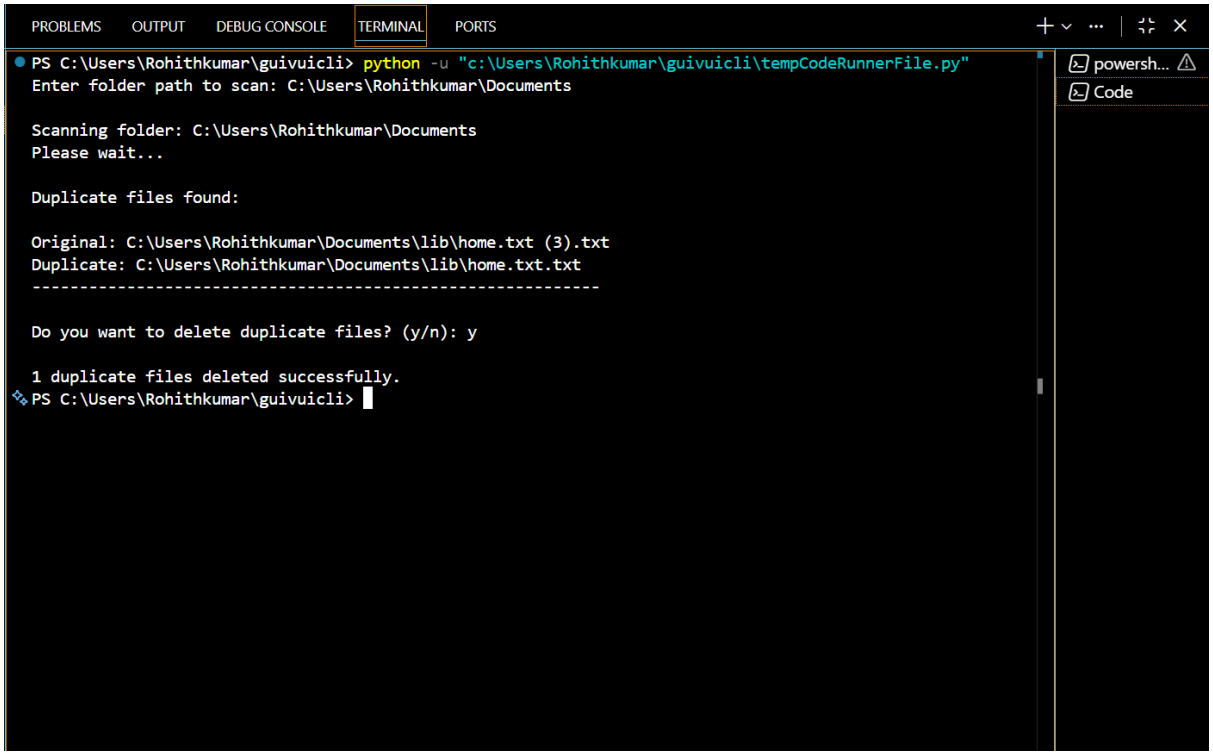
PYTHON CODE:

```
import os
```

```
import hashlib
import sys

def get_file_hash(filepath):
    hasher = hashlib.md5()
    try:
        with open(filepath, 'rb') as f:
            while chunk := f.read(4096):
                hasher.update(chunk)
        return hasher.hexdigest()
    except:
        return None

def find_duplicates(folder_path):
    hashes = {}
    duplicates = {}

    print("\nScanning folder:", folder_path)
    print("Please wait...\n")

    for root_dir, _, files in os.walk(folder_path):
        for file in files:
            path = os.path.join(root_dir, file)
            file_hash = get_file_hash(path)

            if not file_hash:
                continue

            if file_hash in hashes:
```

```
            duplicates.setdefault(file_hash, []).append(path)
        else:
            hashes[file_hash] = path

    return hashes, duplicates

def delete_duplicates(duplicates):
    deleted_count = 0

    for files in duplicates.values():
        for file in files:
            try:
                os.remove(file)
                deleted_count += 1
            except:
                print("Could not delete:", file)

    print(f"\n{deleted_count} duplicate files deleted successfully.")

def main():
    if len(sys.argv) > 1:
        folder_path = sys.argv[1]
    else:
        folder_path = input("Enter folder path to scan: ")

    if not os.path.exists(folder_path):
        print("Invalid folder path.")
        return
```

```
    hashes, duplicates = find_duplicates(folder_path)

    if not duplicates:
        print("No duplicate files found ")
        return

    print("Duplicate files found:\n")

    for file_hash, files in duplicates.items():
        print("Original:", hashes[file_hash])
        for dup in files:
            print("Duplicate:", dup)
        print("-" * 60)

    choice = input("\nDo you want to delete duplicate files? (y/n): ").lower()

    if choice == 'y':
        delete_duplicates(duplicates)
    else:
        print("Deletion cancelled.")

if __name__ == "__main__":
    main()
```



2.GRAPHICAL USER INTERFACE:

Description:

The GUI version provides a user-friendly environment where users can perform duplicate file detection without needing technical knowledge. Built using Tkinter, the interface includes buttons, dialog boxes, and file selection windows.

Functionality:

- Allows users to select folders using a file browser.
- Displays scanning status messages.
- Lists original and duplicate files in an organized format.
- Enables deletion through confirmation dialog boxes.
- Prevents accidental deletion by asking user permission.
- Provides visual feedback throughout the process.

Advantage:

The GUI improves usability and is ideal for non-technical users due to its intuitive design.

```
import tkinter as tk

from tkinter import filedialog, messagebox

import os
```

```
import hashlib

root = tk.Tk()
root.title("Duplicate File Remover")
root.geometry("750x500")
root.resizable(False, False)

current_folder = ""
duplicates = {}

def get_file_hash(filepath):
    hasher = hashlib.md5()

    try:
        with open(filepath, 'rb') as f:
            while chunk := f.read(4096):
                hasher.update(chunk)
        return hasher.hexdigest()
    except:
        return None

def scan_duplicates():
    global duplicates

    if not current_folder:
        messagebox.showwarning("Warning", "Please select a folder first!")
        return

    status_label.config(text="Scanning... Please wait.")
```

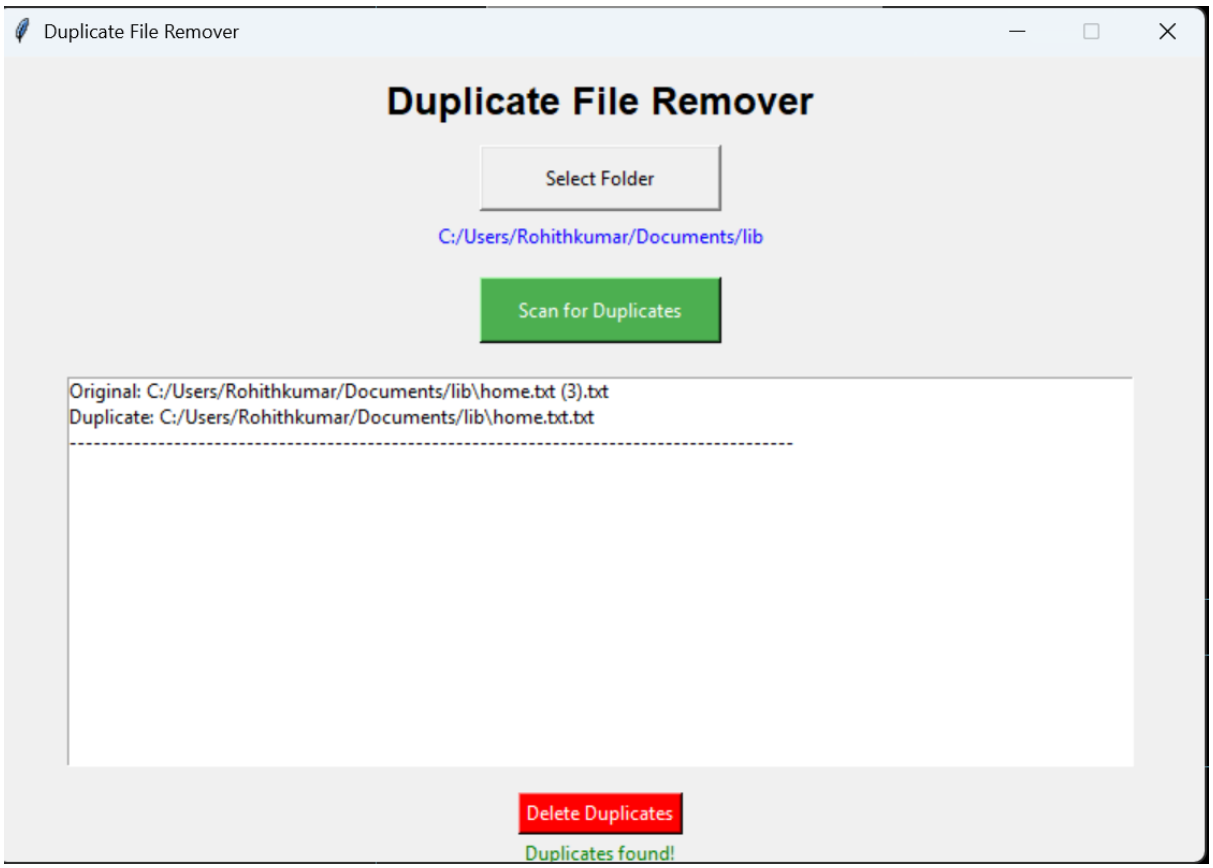
```
root.update()

hashes = {}
duplicates.clear()
file_list.delete(0, tk.END)

for root_dir, _, files in os.walk(current_folder):
    for file in files:
        path = os.path.join(root_dir, file)
        file_hash = get_file_hash(path)

        if not file_hash:
            continue

        if file_hash in hashes:
            duplicates.setdefault(file_hash, []).append(path)
        else:
            hashes[file_hash] = path
if not duplicates:
    file_list.insert(tk.END, "No duplicate files found")
    delete_btn.config(state=tk.DISABLED)
    status_label.config(text="Scan complete. No duplicates.")
    return

for file_hash, files in duplicates.items():
    original = hashes[file_hash]
    file_list.insert(tk.END, f"Original: {original}")

    for dup in files:
```

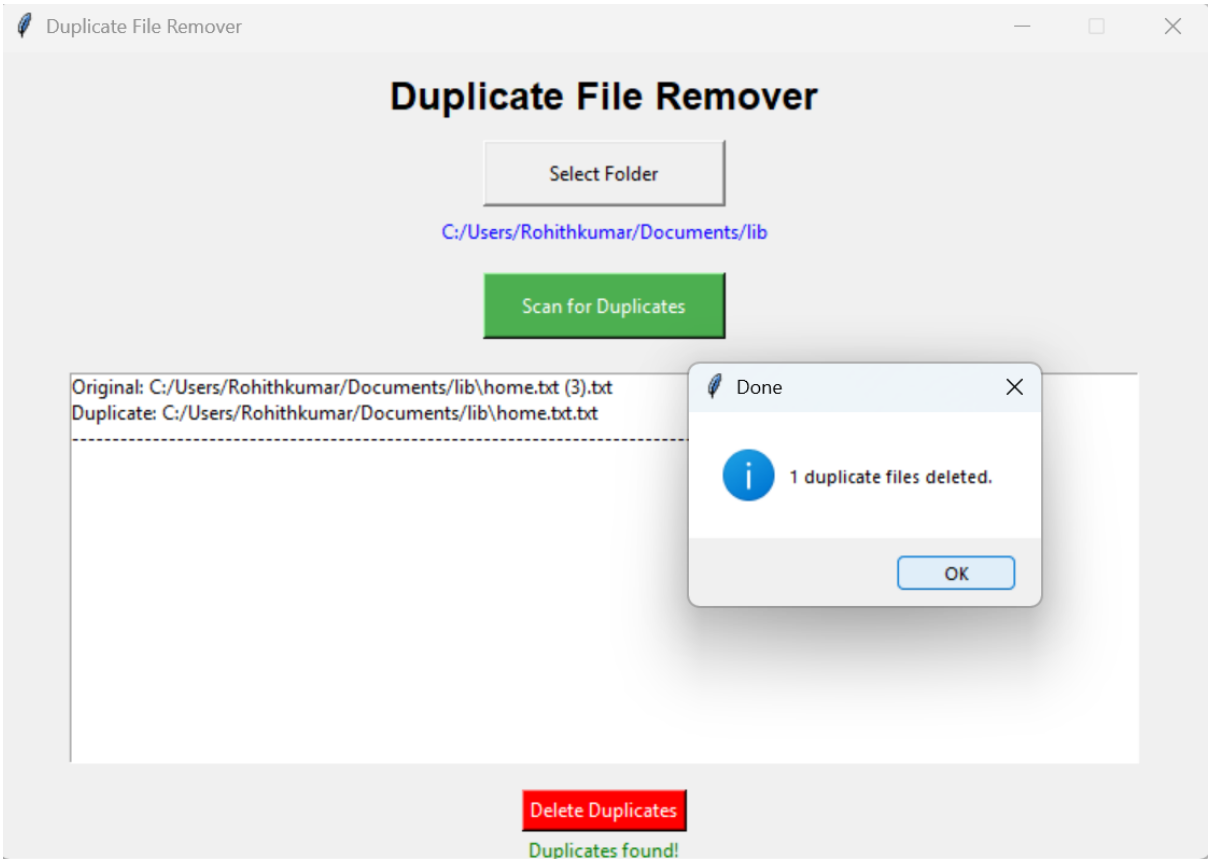
```
            file_list.insert(tk.END, f"Duplicate: {dup}")

        file_list.insert(tk.END, "-"*90)

    delete_btn.config(state=tk.NORMAL)
    status_label.config(text="Duplicates found!")

def delete_duplicates():
    confirm = messagebox.askyesno(
        "Confirm Delete",
        "Delete ALL duplicate files? (Original files will NOT be removed)"
    )

    if not confirm:
        return

    deleted_count = 0

    for files in duplicates.values():
        for file in files:
            try:
                os.remove(file)
                deleted_count += 1
            except:
                pass

    messagebox.showinfo("Done", f"{deleted_count} duplicate files deleted.")
    file_list.delete(0, tk.END)
    delete_btn.config(state=tk.DISABLED)
```

```
    status_label.config(text="Cleanup completed.")

def browse_folder():
    global current_folder

    folder = filedialog.askdirectory()
    if folder:
        current_folder = folder
        folder_label.config(text=folder)
        status_label.config(text="Folder selected. Ready to scan.")

title = tk.Label(
    root,
    text="Duplicate File Remover",
    font=("Arial", 18, "bold")
)
title.pack(pady=10)

browse_btn = tk.Button(
    root,
    text="Select Folder",
    width=20,
    height=2,
    command=browse_folder
)
browse_btn.pack()

folder_label = tk.Label(
    root,
```

```
    text="No folder selected",
    fg="blue",
    wraplength=650
)
folder_label.pack(pady=5)

scan_btn = tk.Button(
    root,
    text="Scan for Duplicates",
    width=20,
    height=2,
    bg="#4CAF50",
    fg="white",
    command=scan_duplicates
)
scan_btn.pack(pady=10)

file_list = tk.Listbox(
    root,
    width=110,
    height=15
)
file_list.pack(pady=10)

delete_btn = tk.Button(
    root,
    text="Delete Duplicates",
    bg="red",
    fg="white",
```

```
    state=tk.DISABLED,
    command=delete_duplicates
)
delete_btn.pack(pady=5)

status_label = tk.Label(
    root,
    text="Select a folder to begin.",
    fg="green"
)
status_label.pack()

root.mainloop()
```





3.VOICE USER INTERFACE:

Description:

The VUI version enables hands-free interaction using speech recognition technology. Users can initiate scanning and deletion through voice commands, improving accessibility and modernizing the user experience.

Functionality:

- Accepts voice commands such as “scan downloads”.
- Detects folder names within commonly used directories.
- Uses ambient noise adjustment for better speech accuracy.
- Converts speech to text using a recognition engine.
- Performs duplicate detection using the same hashing technique.
- Requests voice confirmation before deleting files.
- Supports natural language interaction.

Advantage:

The VUI enhances accessibility and demonstrates how voice technology can simplify system operations.

```
import os
import hashlib
import speech_recognition as sr


def listen_command(prompt):
    print(prompt)

    recognizer = sr.Recognizer()

    try:
        with sr.Microphone() as source:



            recognizer.adjust_for_ambient_noise(source, duration=1)

            print("Listening...")
            audio = recognizer.listen(
                source,
                timeout=12,
                phrase_time_limit=8
            )

        command = recognizer.recognize_google(audio).lower()
        print("Command received:", command)
        return command

    except sr.WaitTimeoutError:
```

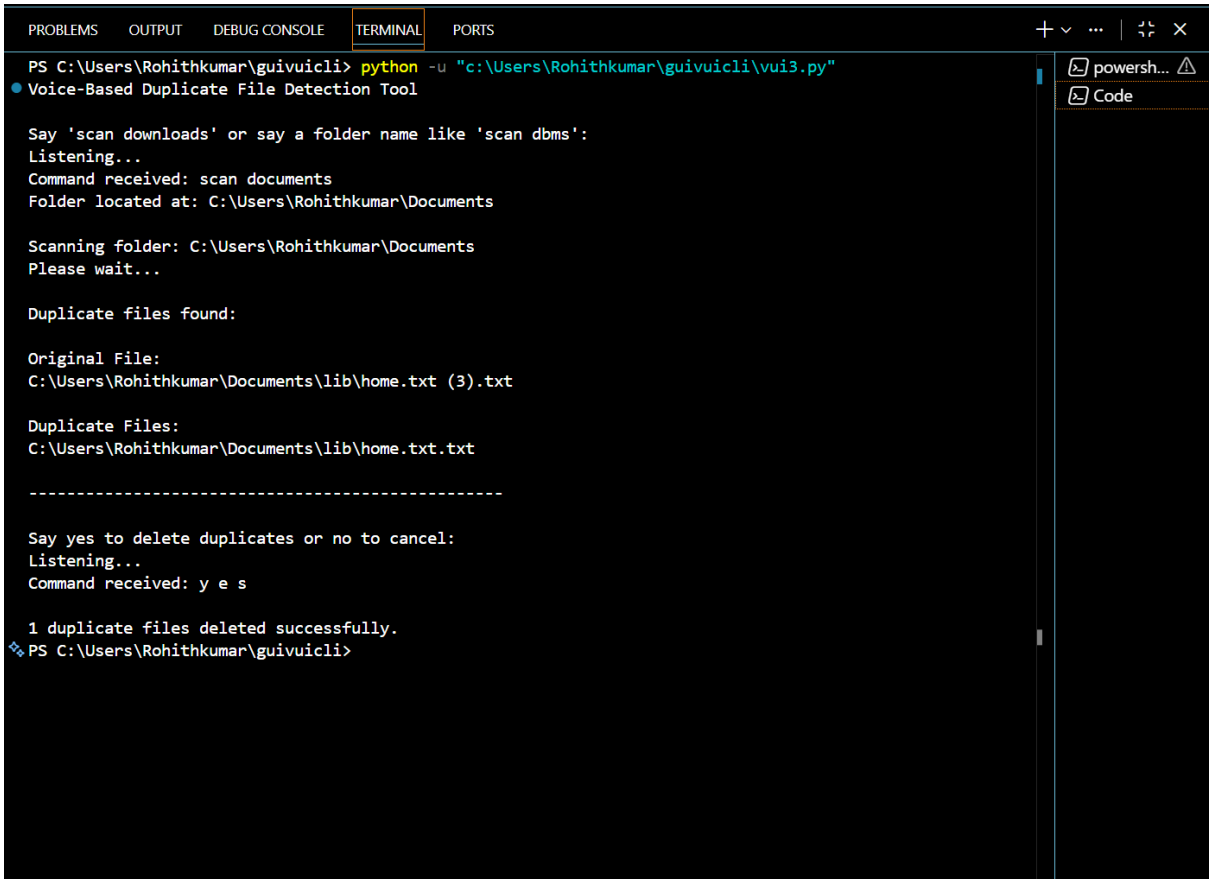
```
        print("No speech detected. Try speaking immediately.")
        return ""

    except sr.UnknownValueError:
        print("Could not understand audio.")
        return ""

    except Exception as e:
        print("Microphone error:", e)
        return ""

def find_folder_by_name(folder_name):

    home = os.path.expanduser("~")

    base_folders = [
        os.path.join(home, "Downloads"),
        os.path.join(home, "Desktop"),
        os.path.join(home, "Documents")
    ]

    folder_name = folder_name.lower()

    for base in base_folders:
        if folder_name in base.lower():
            return base
    for base in base_folders:

        if not os.path.exists(base):
```

```
            continue

        for root, dirs, _ in os.walk(base):
            for d in dirs:
                if folder_name in d.lower():
                    return os.path.join(root, d)

    return None

def get_file_hash(filepath):

    hasher = hashlib.md5()

    try:
        with open(filepath, 'rb') as f:
            while chunk := f.read(4096):
                hasher.update(chunk)

        return hasher.hexdigest()

    except:
        return None

def find_duplicates(folder_path):

    hashes = {}
    duplicates = {}

    print("\nScanning folder:", folder_path)
```

```
    print("Please wait...\n")

    for root_dir, _, files in os.walk(folder_path):
        for file in files:

            path = os.path.join(root_dir, file)
            file_hash = get_file_hash(path)

            if not file_hash:
                continue

            if file_hash in hashes:
                duplicates.setdefault(file_hash, []).append(path)
            else:
                hashes[file_hash] = path

    return hashes, duplicates

def delete_duplicates(duplicates):

    deleted_count = 0

    for files in duplicates.values():
        for file in files:
            try:
                os.remove(file)
                deleted_count += 1
            except:
                print("Could not delete:", file)
```

```
    print(f"\n{deleted_count} duplicate files deleted successfully.")

def main():

    print("Voice-Based Duplicate File Detection Tool\n")

    command = listen_command(
        "Say 'scan downloads' or 'scan documents' to begin:"
    )

    if "scan" not in command:
        print("Command not recognized.")
        return

    folder_name = command.replace("scan", "").strip()

    folder_path = find_folder_by_name(folder_name)

    if not folder_path:
        print("Folder not found.")
        return

    print("Folder located at:", folder_path)

    hashes, duplicates = find_duplicates(folder_path)

    if not duplicates:
        print("No duplicate files found.")
```

```
        return

    print("Duplicate files found:\n")

    for file_hash, files in duplicates.items():

        print("Original File:")
        print(hashes[file_hash])

        print("\nDuplicate Files:")
        for dup in files:
            print(dup)

        print("\n" + "-" * 50 + "\n")

    confirm = listen_command(
        "Say 'yes' to delete duplicates or 'no' to cancel:"
    )

    if "yes" in confirm:
        delete_duplicates(duplicates)
    else:
        print("Operation cancelled.")

if __name__ == "__main__":
    main()
```



User Satisfaction

User satisfaction was evaluated by comparing the ease of use, efficiency, and interaction quality of the three interfaces.

- The **CLI** was found to be the fastest and most resource-efficient but required technical knowledge.
- The **GUI** provided the highest level of usability due to its visual guidance and structured layout.
- The **VUI** offered an innovative experience by enabling hands-free control, making it useful for accessibility-focused environments.

Overall, users preferred the GUI for everyday use, while CLI was favored for speed, and VUI was appreciated for its modern interaction style.

Conclusion

The Multi-Interface Duplicate File Detection System successfully demonstrated how the same functionality can be implemented across different user interfaces while maintaining consistent performance.

Through this experiment, it was observed that interface design plays a significant role in user experience. The CLI offers speed and efficiency, the GUI enhances usability, and the VUI introduces intelligent interaction through voice commands.

This project highlights the importance of selecting the appropriate interface based on user requirements and application context. Implementing multiple interfaces not only improves accessibility but also strengthens software adaptability for diverse users.